EVM 바이트코드 인터프리터 분석

개념적 배경

Ethereum Virtual Machine(EVM)은 스택 기반의 가상 머신으로, 스마트 컨트랙트의 바이트코드를 실행합니다. 바이트코드 인터프리터는 EVM의 핵심 구성 요소로, 각 opcode를 순차적으로 해석하고 실행하는 역할을 담당합니다.

설계 철학

1. **스택 기반 아키텍처**: 레지스터 기반 머신보다 구현이 단순하고 검증 가능
2. **결정론적 실행**: 동일한 입력에 대해 항상 동일한 결과 보장
3. **가스 기반 실행 제어**: 무한 루프 방지와 리소스 사용량 제한
4. **점프 테이블 최적화**: 빠른 opcode 디스패치를 위한 구조

구현 분석

핵심 데이터 구조

EVMInterpreter 구조체

```
type EVMInterpreter struct {
evm *EVM // EVM 인스턴스 참조
table *JumpTable // opcode 점프 테이블
hasher crypto.KeccakState // Keccak256 해시 인스턴스 (재사용)
hasherBuf common.Hash // 해시 결과 버퍼 (재사용)
```

```
readOnly bool // 상태 변경 금지 플래그
returnData []byte // 마지막 CALL의 반환 데이터
}
```

ScopeContext 구조체

```
type ScopeContext struct {
Memory *Memory // EVM 메모리
Stack *Stack // EVM 스택
Contract *Contract // 실행 중인 컨트랙트
}
```

점프 테이블 시스템

점프 테이블은 각 opcode에 대한 실행 함수와 메타데이터를 저장하는 배열입니다:

```
type operation struct {
execute executionFunc // 실행 함수
constantGas uint64 // 고정 가스 비용
dynamicGas gasFunc // 동적 가스 계산 함수
minStack int // 최소 스택 요구사항
maxStack int // 최대 스택 제한
memorySize memorySizeFunc // 메모리 크기 계산 함수
undefined bool // 정의되지 않은 명령어 여부
```

```
}
```

하드포크별 점프 테이블

geth는 각 하드포크마다 별도의 점프 테이블을 유지합니다:

- `frontierInstructionSet` : 초기 Ethereum 명령어 세트
- `homesteadInstructionSet` : Homestead 하드포크 추가 명령어
- `byzantiumInstructionSet` : Byzantium 하드포크 (STATICCALL, RETURNDATASIZE 등)
- `constantinopleInstructionSet` : Constantinople 하드포크 (SHL, SHR, SAR, CREATE2 등)
- `istanbulInstructionSet` : Istanbul 하드포크 (ChainID, SELFBALANCE 등)
- `berlinInstructionSet` : Berlin 하드포크 (EIP-2929 가스 변경)
- `londonInstructionSet` : London 하드포크 (EIP-1559, BASEFEE)
- `shanghaiInstructionSet` : Shanghai 하드포크 (PUSH0)
- `cancunInstructionSet` : Cancun 하드포크 (BLOBHASH, MCOPY 등)

실행 흐름

메인 실행 루프

`EVMInterpreter.Run()` 메서드는 바이트코드 실행의 핵심입니다:

```
func (in *EVMInterpreter) Run(contract *Contract, input []byte, readOnly  bool)

// 1. 호출 깊이 증가 (1024 제한)

in.evm.depth++

defer  func() { in.evm.depth-- }()
```

```
// 2. readOnly 플래그 설정
if readOnly && !in.readOnly {
in.readOnly = true
defer func() { in.readOnly = false }()
}
// 3. 이전 호출의 반환 데이터 초기화
in.returnData = nil
// 4. 빈 코드 체크
if len(contract.Code) == 0 {
return nil, nil
}
// 5. 실행 컨텍스트 초기화
var (
op OpCode
jumpTable = in.table
mem = NewMemory()
stack = newstack()
callContext = &ScopeContext{
Memory: mem,
Stack: stack,
Contract: contract,
}
pc = uint64(0) // 프로그램 카운터
```

```
cost uint64 // 가스 비용
)
// 6. 메인 실행 루프
for {
// opcode 가져오기
op = contract.GetOp(pc)
operation := jumpTable[op]
// 스택 검증
if sLen := stack.len(); sLen < operation.minStack {
return nil, &ErrStackUnderflow{stackLen: sLen, required: operation.minStack}
} else if sLen > operation.maxStack {
return nil, &ErrStackOverflow{stackLen: sLen, limit: operation.maxStack}
}
// 가스 소모
cost = operation.constantGas
if contract.Gas < cost {
return nil, ErrOutOfGas
}
contract.Gas -= cost
// 동적 가스 계산
if operation.dynamicGas != nil {
dynamicCost, err := operation.dynamicGas(in.evm, contract, stack, mem, memorySiz
cost += dynamicCost
```

```
if err != nil || contract.Gas < dynamicCost {
return nil, ErrOutOfGas
}
contract.Gas -= dynamicCost
}
// 메모리 확장
if memorySize > 0 {
mem.Resize(memorySize)
}
// opcode 실행
res, err = operation.execute(&pc, in, callContext)
if err != nil {
break
}
pc++
}
return res, err
}
```

스택 검증 메커니즘

각 opcode 실행 전에 스택 상태를 검증합니다:

1. **언더플로우 검사**: 필요한 최소 스택 항목 수 확인

2. **오버플로우 검사**: 스택 크기 제한(1024) 확인

3. **가스 소모**: 스택 검증 후 가스 차감

메모리 관리

메모리는 필요에 따라 동적으로 확장됩니다:

```
// 메모리 크기 계산
if operation.memorySize != nil {
memSize, overflow := operation.memorySize(stack)
if overflow {
return nil, ErrGasUintOverflow
}
// 32바이트 워드 단위로 계산
if memorySize, overflow = math.SafeMul(toWordSize(memSize), 32); overflow {
return nil, ErrGasUintOverflow
}
}
```

점프 테이블과 디스패치 최적화

점프 테이블 구조

점프 테이블은 256개 요소의 배열로, 각 opcode(0x00-0xFF)에 대응하는 operation 구조체를 저장합니다:

```
type JumpTable [256]*operation
```

디스패치 최적화 기법

1. **직접 배열 접근**: `jumpTable[op]` 로 O(1) 시간에 operation 조회
2. **함수 포인터 사용**: 가상 함수 호출 오버헤드 최소화
3. **인라인 최적화**: 자주 사용되는 opcode는 컴파일러가 인라인 처리

opcode 실행 함수 예시

산술 연산 (ADD)

```
func opAdd(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]byt
x, y := scope.Stack.pop(), scope.Stack.peek()
y.Add(&x, y) // y = x + y
return nil, nil
}
```

스택 조작 (DUP1)

```
func makeDup(size int64) executionFunc {
return func(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]by
scope.Stack.dup(int(size))
return nil, nil
}
```

```
}
```

점프 명령 (JUMP)

```
func opJump(pc *uint64, interpreter *EVMInterpreter, scope *ScopeContext) ([]by
if interpreter.evm.abort.Load() {
return nil, errStopToken
}
pos := scope.Stack.pop()
if !scope.Contract.validJumpdest(&pos) {
return nil, ErrInvalidJump
}
*pc = pos.Uint64() - 1 // pc는 루프에서 1 증가하므로 -1
return nil, nil
}
```

성능 최적화

1. 메모리 풀링

스택과 메모리 객체는 sync.Pool을 사용하여 재사용됩니다:

```
var stackPool = sync.Pool{
New: func() interface{} {
```

```
return &Stack{data: make([]uint256.Int, 0, 16)}
},
}

var memoryPool = sync.Pool{
New: func() any {
return &Memory{}
},
}
```

2. 해시 인스턴스 재사용

Keccak256 해시 계산을 위한 인스턴스를 재사용합니다:

```
type EVMInterpreter struct {
hasher crypto.KeccakState // 재사용되는 해시 인스턴스
hasherBuf common.Hash // 재사용되는 결과 버퍼
}
```

3. 점프 테이블 복사 최적화

EIP 활성화 시에만 점프 테이블을 복사합니다:

```
if len(evm.Config.ExtraEips) > 0 {
// Deep-copy jumptable to prevent modification of opcodes in other tables
```

```
table = copyJumpTable(table)

}
```

4. 가스 계산 최적화

- **상수 가스**: 컴파일 타임에 결정되는 고정 비용
- **동적 가스**: 런타임에 계산되는 가변 비용 (메모리 확장, 스토리지 접근 등)

에러 처리 메커니즘

실행 중단 조건

1. **가스 부족**: `ErrOutOfGas`
2. **스택 언더플로우**: `ErrStackUnderflow`
3. **스택 오버플로우**: `ErrStackOverflow`
4. **잘못된 점프**: `ErrInvalidJump`
5. **실행 되돌림**: `ErrExecutionReverted`
6. **쓰기 보호 위반**: `ErrWriteProtection`

정상 종료 조건

1. **STOP**: 정상 종료
2. **RETURN**: 데이터와 함께 정상 종료
3. **SELFDESTRUCT**: 컨트랙트 자기 파괴

실제 사용 예시

간단한 ADD 연산 실행

```
바이트코드: 0x6001600201 (PUSH1 0x01 PUSH1 0x02 ADD)
```

```
실행 과정:
1. PUSH1 0x01: 스택에 1 푸시 -> [1]
2. PUSH1 0x02: 스택에 2 푸시 -> [1, 2]
3. ADD: 스택에서 두 값을 팝하고 합계를 푸시 -> [3]
```

가스 소모 계산

```
PUSH1: 3 가스 (GasFastestStep)
ADD: 3 가스 (GasFastestStep)
총 가스: 9 가스 (PUSH1 × 2 + ADD)
```

관련 시스템

의존성 관계

- **EVM**: 전체 실행 환경 제공
- **StateDB**: 상태 읽기/쓰기 인터페이스
- **Contract**: 실행 중인 컨트랙트 정보
- **Memory**: EVM 메모리 관리
- **Stack**: EVM 스택 관리

상호작용 패턴

1. **EVM → Interpreter**: 바이트코드 실행 요청
2. **Interpreter → JumpTable**: opcode 조회
3. **Interpreter → Stack/Memory**: 데이터 조작
4. **Interpreter → StateDB**: 상태 변경 (SLOAD, SSTORE 등)

영향 범위

- **트랜잭션 실행**: 모든 스마트 컨트랙트 호출
- **가스 소모**: 각 opcode의 가스 비용 결정
- **상태 변경**: 블록체인 상태 업데이트
- **실행 추적**: 디버깅과 모니터링

이 바이트코드 인터프리터는 Ethereum의 핵심 실행 엔진으로, 모든 스마트 컨트랙트의 실행을 담당하며, 성능과 보안을 모두 고려한 정교한 설계를 보여줍니다.